arXiv:2607.14656v2 [cs.DS] 20 Jul 2026

Semi-Streaming Matching in a Single Pass II: Greedy is Optimal

Sepehr Assadi* Max Jiang* Mars Xiang*

Abstract

We prove that no single-pass semi-streaming algorithm (deterministic or randomized) can achieve a better-than-half approximation to the maximum matching problem. This implies the optimality of the naive greedy algorithm, answering an outstanding open question in the graph streaming literature since the introduction of the model over two decades ago.

Our proof follows the "blueprint framework" introduced previously by the authors, which reduced proving lower bounds for semi-streaming matching to constructing certain combinatorial objects called blueprints. We present an optimal construction of blueprints that when used in this framework implies our semi-streaming matching lower bound.

Our results also imply that the optimal competitive ratio of online matching with preemption is half, again matching the naive greedy algorithm, settling this open question as well.

*(sepehr@assadi.info, max.jiang@uwaterloo.ca, mxiang@uwaterloo.ca) School of Computer Science, University of Waterloo. Supported in part by an NSERC Discovery Grant, a Faculty of Math Research Chair grant, and an Undergraduate Research Fellowship (URF) from University of Waterloo.
Part of this work was done while Mars Xiang was at The Interaction Company of California.

Contents

1 Introduction **1**
1.1 Blueprint Framework of [AJX26] 1
1.2 Our Blueprint Construction at a High Level 3
1.3 Further Corollaries of Our Result 5

2 Preliminaries **6**

3 Random Walk on a Line with Symmetric Maxima **7**
3.1 Proof of Lemma 3.1 8

4 The Blueprint Construction **10**
4.1 Setup 10
4.2 The Construction 13
4.3 Value Calculation 14

A *Upper Bounding* the Value of Blueprints **20**

1 Introduction

In the semi-streaming model for graph problems, formalized by [FKM+05], we have an n-vertex graph $G = (V, E)$ whose edges are given to the algorithm in some arbitrarily ordered stream; the algorithm makes one or a few passes over the stream, uses $\widetilde{O}(n) := O(n \cdot \text{polylog}(n))$ memory, and at the end of the stream, outputs a solution to the problem at hand. In this paper, we study **single-pass** semi-streaming algorithms for the **maximum matching** problem.

Alongside the introduction of the model, [FKM+05] observed a simple (1/2)-approximation for this problem: greedily maintain a maximal matching of the arriving edges. The two decades since then have witnessed a high throughput of results on the semi-streaming matching problem, and yet, this simple algorithm remains the state of the art. A handful of impossibility results have also been proven over the years, ruling out approximation ratios of $(2/3)$ [GKK12], $(1-1/e) \sim 0.632$ [Kap13], $(1+\ln(2))^{-1} \sim 0.590$ [Kap21], and most recently $(8-2\sqrt{10})/3 \sim 0.558$ [AJX26].

In light of this state of affairs, whether the half-approximation of the greedy algorithm can be improved has been an outstanding open question since the introduction of the model [FKM+05, KMM12, McG13, McG14, KT17, KN21]. Over the years, this question has been commonly referred to as "major" [Tir18, GKMS19, GKRS20, FMU22], "central" [ABKL23, DGR25], "key" [ADKN23], "main" [Ber24], one of the "most longstanding" [Beh21, AB21, DGR25] or "most appealing" [EHM16, CJMM17], "perhaps the most well-known" [Gam21], and "the most basic" [FS22] of open questions in the area. It has also been more affectionally called "vexing" [CK14], "baffling" [KT17], "most tantalizing" [Waj21], "notorious" [FNSZ22], and "elusive" [Nai24], among others.

We fully resolve this question in our work.

Theorem 1. *There is no single-pass semi-streaming algorithm for the maximum bipartite matching problem that can achieve any constant approximation ratio strictly better than half with constant probability.*

Theorem 1 settles the complexity of the semi-streaming matching problem: after all, the naive greedy algorithm turns out to be optimal for this fundamental problem.

Our proof of Theorem 1 is based on the *"blueprint framework"* of the recent work of the authors in [AJX26]. We describe this framework next and put our result in this context.

1.1 Blueprint Framework of [AJX26]

In [AJX26], we reduced proving lower bounds for semi-streaming matching to constructing "constant-size" (relative to stream length) combinatorial objects called *blueprints*. We now define blueprints and this framework[1] (following [AJX26], we use 'blackboard bold' letters for blueprint notation).

Definition 1.1. For integers $P, C \geqslant 1$, we define a **blueprint** with parameters P, C as any bipartite graph $\mathbb{G} = (\mathbb{L}, \mathbb{R}, \mathbb{E})$ such that:

- Vertices in $\mathbb{L}$ and $\mathbb{R}$ are identified by tuples in $[C]^P$; specifically, for any $x \in [C]^P$ we have a vertex $\mathbb{L}(x) \in \mathbb{L}$ and a vertex $\mathbb{R}(x) \in \mathbb{R}$.
- Edges of $\mathbb{G}$ are partitioned into P groups: $\mathbb{E} = \mathbb{E}^{(1)} \sqcup \cdots \sqcup \mathbb{E}^{(P)}$.

[1] We directly define what [AJX26] calls a *simple* blueprint as the blueprint itself, as it is a lot easier to describe and we do not need the extra power of the more general definition.

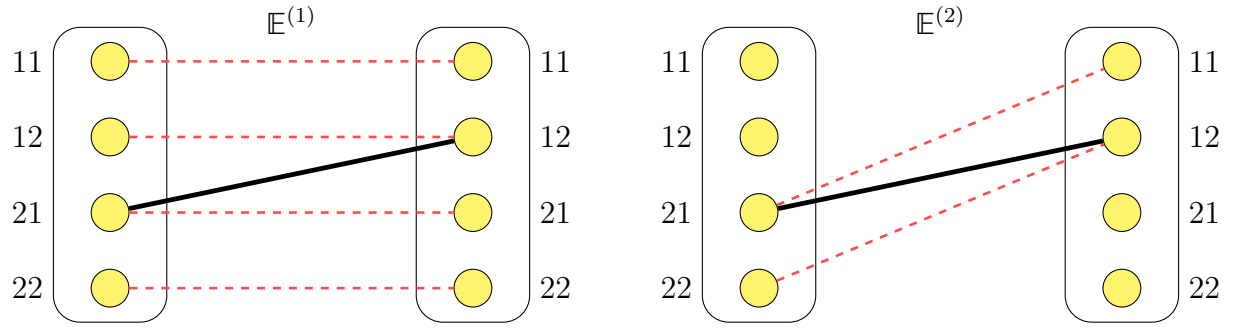
Definition 1.1 simply gives the "skeleton" of a blueprint; what we really need from a blueprint is to satisfy the following two constraints (see Section 2 for a quick review of notation).

Definition 1.2. We say a blueprint $\mathbb{G} = (\mathbb{L}, \mathbb{R}, \mathbb{E})$ with parameters (P, C) is **proper** if it satisfies the following constraints:

- **Matching constraint:** The edges in $\mathbb{E}$ form a single matching in $\mathbb{G}$.
- **Ban constraints:** For any $x, y \in [C]^P$ and vertices $\mathbb{L}(x)$ and $\mathbb{R}(y)$ incident on the same edge $\mathbb{e} = (\mathbb{L}(x), \mathbb{R}(y)) \in \mathbb{E}^{(p)}$ for some $p \in [P]$:

  for any $z \in [C]^{P-p+1}$, at least one of vertices $\mathbb{L}(x_{<p} \circ z)$ or $\mathbb{R}(y_{<p} \circ z)$ has no edges in $\mathbb{G}$; we say these two vertices are **banned together** by the edge $\mathbb{e}$.

See Figure 1 for an example of ban constraints.



(a) Adding any edge in $\mathbb{E}^{(1)}$ bans the pairs in the identity permutation in the entire blueprint.

(b) If the same edge was instead added to $\mathbb{E}^{(2)}$, it would ban fewer but different pairs.

Figure 1: An illustration of ban constraints in a blueprint with $P = C = 2$ with labels of vertices drawn next to them. Solid (black) edges show two possible edges in two different blueprints and dashed (red) edges show banned pairs resulting from inclusion of each of these edges separately.

Finally, the following quantity determines how "good" a blueprint is for our purpose.

Definition 1.3. Define the **value** of a blueprint $\mathbb{G} = (\mathbb{L}, \mathbb{R}, \mathbb{E})$ as:

$$value(\mathbb{G}) := \frac{|\mathbb{E}|}{|\mathbb{L}|} = \frac{|\mathbb{E}|}{|\mathbb{R}|} = \frac{|\mathbb{E}|}{C^P}.$$

We refer the reader to [AJX26] for several examples of blueprints and the intuition (and the formal proof) behind the connection between blueprints and semi-streaming matching. For us, we need the following theorem that captures the blueprint framework put forward in [AJX26][2].

Theorem 2 ([AJX26])**.** *Suppose there is a proper blueprint $\mathbb{G}$ and define $\alpha := \frac{2-2\cdot value(\mathbb{G})}{2-value(\mathbb{G})}$. Then, for any fixed $\delta > 0$, and any sufficiently large n, as a function of $\mathbb{G}$ and δ, the following holds. Any single-pass semi-streaming algorithm for maximum bipartite matching on n-vertex graphs cannot achieve an $(\alpha + \delta)$-approximation with probability more than $1 - \delta$.*

[2]In light of what we said earlier about *simple* blueprints in [AJX26], it is worth emphasizing that [AJX26] proves this theorem for simple blueprints and then shows simple and not-simple blueprints are equivalent; given our construction is already a simple blueprint, we do not need this equivalence and defined simple blueprints as blueprints.

In [AJX26], we constructed blueprints with values converging to $(5-\sqrt{10})/3$ which, plugged in Theorem 2, proves the impossibility of better than $(8-2\sqrt{10})/3 \sim 0.558$ approximation for the semi-streaming matching problem. The limit of the framework of [AJX26] however is determined by the following quantity

$$\sup\{value(\mathbb{G}) : \mathbb{G} \text{ a proper blueprint}\}, \tag{1}$$

which was not well understood in [AJX26].

Our blueprint construction. We determine the optimal value of Eq (1) in this work, proving the following, which is the main *technical* result of our work.

Theorem 3. *There is a sequence of proper blueprints $(\mathbb{G}_m)_{m\geqslant 1}$ with parameter $P_m \to \infty$ and*

$$\lim_{m\to\infty} value(\mathbb{G}_m) = 2/3.$$

Plugging in the blueprints in Theorem 3 in Theorem 2 then immediately proves Theorem 1. Our proof of Theorem 3 is constructive and we give an explicit description of the blueprints in the family (even though this is not required to apply Theorem 2). We also note that as pointed out in [AJX26], one does need $P \to \infty$ for the value of the blueprint to be able to reach 2/3.

As an aside, we should note that, technically speaking, one also needs to prove an upper bound on Eq (1) to fully determine the value of Eq (1) (although this is not needed for proving Theorem 1). However, a 2/3 upper bound follows immediately from Theorem 2 and the greedy algorithm for the semi-streaming matching problem. We give a different and direct proof of this upper bound in Section A as it provides further insight into the structure of blueprints.

1.2 Our Blueprint Construction at a High Level

Blueprints in Theorem 3 are created by using elementary properties of random walks on the integer line. The general idea of the construction is as follows (as our blueprints are *entirely disjoint* from [AJX26] technique-wise, we will not review those blueprints and refer the reader to [AJX26]).

Vertices and random walks. Let $m \geqslant 1$ be an integer and $\mathbb{G} = (\mathbb{L}, \mathbb{R}, \mathbb{E})$ be a blueprint with parameters P and C that depend on m. To any label $x \in [C]^P$, we assign a walk $W(x)$ on $\{0, \ldots, 2m\}$ that starts from some state in $[2m-1]$ and in each step moves to either a one larger or one smaller state and terminates when it reaches 0 or $2m$. Define the *reflection* walk $\overline{W}(x)$ that starts at the same state as $W(x)$ but in every step, takes the opposite direction as $W(x)$. This assignment is done so that for x chosen uniformly from $[C]^P$, $W(x)$ (also $\overline{W}(x)$) becomes a random walk whose starting state is sampled from some distribution μ and each of its step is chosen uniformly. This is done by using the first index of $[P]$ to simulate sampling from μ (by partitioning $[C]$ into intervals of lengths proportional to $\mu(i)$-values) and remaining indices to simulate the random walk (by partitioning $[C]$ into two equal halves); see Figure 2 for a quick illustration (we have skipped one other coordinate in $[P]$ called "delay" and will explain it later).

Edges between 0-reaching random walks. Let $\mathbb{L}^\star$ (resp. $\mathbb{R}^\star$) be vertices $\mathbb{L}(x)$ (resp. $\mathbb{R}(x)$) whose random walk $W(x)$ (resp. $\overline{W}(x)$) reaches 0. Moreover, for any $j \in [2m-1]$ and time $t \in [P-1]$, let $\mathbb{L}^\star_{j,t}$ denote the following subset of $\mathbb{L}^\star$:

- All $\mathbb{L}(x) \in \mathbb{L}$ such that $W(x)$ reaches 0, $W(x)$ has j as the *maximum* state it ever visits, and, it has visited j for the *first* time at the t-th step of the walk.

We define $\mathbb{R}^\star_{j,t}$ analogously using the reflection walk $\overline{W}$. In the blueprint, we add a *matching* between $\mathbb{L}^\star_{j,t}$ and $\mathbb{R}^\star_{2m-j,t}$ in $\mathbb{E}^{(t+1)}$ with size equal to the smaller of the two.

Properness of the blueprint. We need to ensure the inserted edges form a matching and that the ban constraints are satisfied. The former is easy to verify as $\mathbb{L}^\star_{j,t}, \mathbb{R}^\star_{j,t}$ sets are disjoint for different j, t and each is incident on a matching. We now prove the ban constraint.

Consider any edge between $\mathbb{L}(x) \in \mathbb{L}^\star_{j,t}$ and $\mathbb{R}(y) \in \mathbb{R}^\star_{2m-j,t}$ in some $\mathbb{E}^{(t+1)}$ for $j \in [2m-1]$ and $t \in [P-1]$ (all edges are of this form). The ban constraint requires that for any $z \in [C]^{P-t}$, at most one of $\mathbb{L}(x_{\leqslant t} \circ z)$ or $\mathbb{R}(y_{\leqslant t} \circ z)$ is matched. Fixing $x_{\leqslant t}$ and $y_{\leqslant t}$ "brings" any walk of $W(x_{\leqslant t} \circ z)$ and $\overline{W}(y_{\leqslant t} \circ z)$ to the states j and $2m-j$ at their t-th step (by the definitions of $\mathbb{L}^\star_{j,t}, \mathbb{R}^\star_{2m-j,t}$). Now, suppose $\mathbb{L}(x_{\leqslant t} \circ z)$ is matched. Then, it means that the remainder of the walk, which with a slight abuse of notation we denote by $W(z)$, will move from the state j, uses z, and reaches 0 (so $\mathbb{L}(x_{\leqslant t} \circ z)$ becomes matched). But then, the reflection walk $\overline{W}(z)$ will move in the "opposite" direction: it moves from $2m-j$, uses z, and thus reaches $2m$ instead; this means that $\mathbb{R}(y_{\leqslant t} \circ z)$ will not be matched, as desired. This is the entire proof of the ban constraint (see also Figure 3).

Value calculation. The final step is to consider the value of this blueprint. In our construction, we are going to ensure that the sizes of each of $\mathbb{L}^\star_{j,t}$ and $\mathbb{R}^\star_{2m-j,t}$ that are matched together are (almost) equal to each other. This will ensure that the size of the matching of the blueprint is (almost) $|\mathbb{L}^\star|$, which implies that

$$value(\mathbb{G}) := \frac{|\mathbb{L}^\star|}{C^P} = \Pr_{x \in_R [C]^P} (\text{walk } W(x) \text{ reaches } 0)\,. \tag{2}$$

As stated earlier, the constraint we have is that $\mathbb{L}^\star_{j,t}$ and $\mathbb{R}^\star_{2m-j,t}$ sets should have roughly the same size. For now let us *relax* this constraint to instead ask that $\mathbb{L}^\star_j := \cup_t \mathbb{L}^\star_{j,t}$ and $\mathbb{R}^\star_{2m-j} := \cup_t \mathbb{R}^\star_{2m-j,t}$ have the same size. This translates to having,

$$\begin{aligned} \forall j \in [2m-1] \Pr_{x \in_R [C]^P} &(\text{walk } W(x) \text{ reaches } 0 \text{ and maximum state it visits is } j) \\ &= \Pr_{x \in_R [C]^P} \left(\text{walk } \overline{W}(x) \text{ reaches } 0 \text{ and maximum state it visits is } 2m-j\right). \end{aligned} \tag{3}$$

The underlying distributions of both $W(x)$ and $\overline{W}(x)$ here are just standard random walks starting from a random state chosen according to some distribution μ. Thus, at this point, the goal is to maximize Eq (2) subject to the constraints in Eq (3), by choosing the initial distribution μ carefully.

The type of random walk we have here is often called "gambler's ruin" and is well studied as one of the simplest examples of Markov Chains; see, e.g. [LP26, Section 2.1]. For instance, it is known that starting from a *uniform* distribution μ, maximum visited state will also be distributed uniformly and thus certainly satisfies Eq (3). But in this case, the walk has an equal chance of reaching 0 and $2m$, making $value(\mathbb{G}) = 1/2$ only. However, by "shifting" the distribution of μ towards 0, while still enforcing Eq (3), one can improve the objective function. We are not aware of prior work that study this particular formulation but solving this optimization problem has a standard method: one can cast the problem as a linear program in $\mu(i)$-values and obtain a closed form formula for the variables. Doing this in our case leads to the objective value of $2m/(3m+1)$ (see Lemma 3.1), giving us the desired value for the blueprint.

There is however one catch with the argument above: we moved from enforcing (almost) equality in sizes of $\mathbb{L}^\star_{j,t}$ and $\mathbb{R}^\star_{2m-j,t}$ to only $\mathbb{L}^\star_j$ and $\mathbb{R}^\star_{2m-j}$; but, symmetrizing the distribution of maxima (through the choice of μ) does not mean the time to reaching them will also be symmetric. This however can be handled rather easily by introducing **random delays** in the random walks: we dedicate one more coordinate in $[P]$ for this task, and start the random walks at different time stages to ensure that distribution of "time to hit maximum" becomes (almost) uniform over the time steps which fixes the above problem.

1.3 Further Corollaries of Our Result

We mention two further corollaries of our results. The first one is for the matching problem but in another model, the *online matching with preemption*, whereas the second one is for another problem, the *minimum vertex cover*, but back in the semi-streaming model.

Online matching with preemption. In this problem, the edges of the graph are arriving sequentially and the algorithm must maintain a matching by accepting or rejecting arriving edges and can preempt previously accepted ones by discarding them. This model is a generalization of the original online matching (with one-sided vertex arrival) in [KVV90].

The *competitive ratio* of an algorithm in this model is the ratio of the matching size provided by the algorithm compared to the maximum matching of the graph. The greedy algorithm achieves a half competitive ratio in this model with no need for preemption. When preemptions are not allowed, [GKM+19] proved that half approximation is optimal. No better algorithm is known even when preemption is allowed and a series of results over the years have ruled out competitive ratios of $(1-1/e) \sim 0.632$ [KVV90], $(1+\ln(2))^{-1} \sim 0.590$ [ELSW13], $(2-\sqrt{2}) \sim 0.585$ [HPT+19], and very recently 0.5661 [KS26][3]. We prove that the greedy algorithm is also optimal for this problem.

Corollary 1.4. *There is no algorithm for the online matching problem with preemption with an expected competitive ratio strictly better than half by any constant.*

As already observed in [AJX26], the blueprint framework also implies lower bounds for the online matching problem with preemption (given this connection, technically, the best known bound on the competitive ratios prior to our work is the 0.558 bound of [AJX26], which was obtained independently and concurrently to [KS26]). Thus, Corollary 1.4 directly follows from our Theorem 3 and the blueprint framework of [AJX26].

Semi-streaming (bipartite) vertex cover. Going back to the semi-streaming model, a closely related problem to maximum bipartite matching is its dual, the minimum bipartite vertex cover problem. The greedy algorithm is also a 2-approximation algorithm for this problem and no better algorithms are known. It has been asked in prior work, e.g., [ABB+19, DGR25] whether this algorithm can be improved. We resolve this question in this work.

Corollary 1.5. *There is no single-pass semi-streaming algorithm for the minimum bipartite vertex cover problem that can achieve any constant approximation ratio strictly better than two with constant probability.*

We note that prior to our work, no 2-approximation lower bounds were known for this problem even in general graphs (the UGC hardness of 2-approximation vertex cover [KR03] does not imply a semi-streaming lower bound, as runtime of algorithms is not bounded in this model).

The proof of this result is again by observing that the blueprint framework of [AJX26] also implies lower bounds for the vertex cover problem given the duality of the problems (plus a standard modification for going from matching lower bounds to vertex cover ones as done previously in the literature, e.g., in [AK17, DGR25]).

[3] It is not a coincidence that these numbers and their progressions sound very similar to the ones mentioned for the semi-streaming matching: until [AJX26], all previous semi-streaming matching lower bounds in [GKK12, Kap13, Kap21] were based on generalizing online preemptive matching instances to the semi-streaming model (but using completely white-box arguments); see [Kap21] for a detailed discussion of this topic.

2 Preliminaries

Notation. For integers $1 \leqslant a \leqslant b$, we define $[a:b] := \{a, a+1, \ldots, b\}$ and $[b] := \{1, \ldots, b\}$. For a tuple $x = (x_1, \ldots, x_t)$ and $i \in [2:t]$, we denote $x_{<i} := (x_1, \ldots, x_{i-1})$.

For a random variable X, we sometimes write the subscript X in $\Pr_X(f(X))$ (for some f), to explicitly specify the randomness is coming from X. For a random variable X and a set S, we write $X \in_R S$ to mean X is sampled uniformly at random from S.

Gambler's ruin. We use a standard type of random walks on integer line $\{0, 1, \ldots, 2m\}$ for some integer $m \geqslant 1$, often called the *gambler's ruin*; see, e.g. [LP26, Section 2.1].

The walk $W = w_1, w_2, \cdots$ starts at some integer $w_1 \in [2m-1]$ and in each time $t \geqslant 1$, it updates $w_{t+1} = w_t + 1$ with probability half and $w_{t+1} = w_t - 1$ otherwise. The walk terminates at the first time t when $w_t = 0$ or $w_t = 2m$ and remains in this state. Throughout, by a random walk, we always mean the above type of walk unless explicitly stated otherwise.

Fact 2.1 (c.f. [LP26, Proposition 2.1])**.** *For a random walk W over $\{0, \ldots, 2m\}$ and an integer k in $\{0, \ldots, 2m\}$,*

$$\Pr_W(W \text{ reaches } 0 \text{ before } 2m \mid W \text{ starts at } k) = \frac{2m-k}{2m}.$$

Moreover, the expected length of the walk before terminating is at most m^2.

For a (finite) random walk $W = w_1, \cdots, w_T$, where $w_T \in \{0, 2m\}$ is a terminating state, define:

$$\mathsf{w}_{\max} = \mathsf{w}_{\max}(W) := \max_{t \in [T]} \{w_t\} \quad \text{and} \quad \mathsf{t}_{\max} = \mathsf{t}_{\max}(W) := \min\{t \in [T] : w_t = \mathsf{w}_{\max}(W)\},$$

as the *largest* state seen on the walk and the *first* time the walk sees its eventual largest state.

A corollary of Fact 2.1 is the following.

Claim 2.2. *For a random walk W over $\{0, \ldots, 2m\}$ and any $1 \leqslant i \leqslant j < 2m$,*

$$\Pr_W(W \text{ ends in } 0 \wedge \mathsf{w}_{\max}(W) = j \mid W \text{ starts at } i) = \frac{i}{j \cdot (j+1)}.$$

Proof. For this to happen, we need W to reach j before 0, and once on j, to reach 0 before $j+1$. Given the random walk is memoryless, these two events are independent and thus we have

$$\Pr_W(W \text{ ends in } 0 \wedge \mathsf{w}_{\max}(W) = j \mid W \text{ starts at } i) =$$
$$\Pr_W(W \text{ reaches } j \text{ before } 0 \mid W \text{ starts at } i) \cdot \Pr_W(W \text{ reaches } 0 \text{ before } j+1 \mid W \text{ starts at } j).$$

The first term is about a random walk on $\{0, \ldots, j\}$ starting from i and thus by Fact 2.1 has probability i/j. The second term is about a random walk on $\{0, \ldots, j+1\}$ starting from j and thus by Fact 2.1 has probability $1/(j+1)$. Multiplying these two concludes the proof. ∎

We also have the following standard claim on the length of random walks.

Claim 2.3. *The probability that a random walk W starting from any state does not terminate within $2m^3$ steps is at most 2^{-m}.*

Proof. The expected time for a walk W to terminate, starting from any state, is at most m^2 by Fact 2.1. Thus, by Markov bound, the probability that the walk has not terminated after $2m^2$ steps is at most half. Repeating the argument from current state, the probability that m repetitions, each of length $2m^2$, does not terminate the walk is at most 2^{-m}. ∎

3 Random Walk on a Line with Symmetric Maxima

Fix any integer $m \geqslant 1$. We will be using the gambler's ruin random walk on $\{0, \ldots, 2m\}$ described in Section 2 for our proofs. For our purpose, we also need to specify a distribution over initial states with the following two properties: (1) the distribution of $\mathsf{w}_{\max}(W)$ is symmetric (on the interior states) assuming the start state is sampled from this distribution, and, (2) the probability of terminating in state 0 is maximized. The following lemma finds such a distribution for us (we do not explicitly prove that it maximizes the term in (2) and only calculate its value, which is sufficient for us; however, one can prove it is maximized).

Lemma 3.1. *Consider the assignment $\mu : \{1, \ldots, 2m-1\} \to [0,1]$ such that for $i \in [m]$,*

$$\mu(i) := \gamma_m \cdot \frac{2 \cdot (2m+1)}{(2m-i) \cdot (2m-i+1) \cdot (2m-i+2)} \quad \text{for} \quad \gamma_m := \frac{2m \cdot (m+1)}{3m+1},$$

and for $i \in [m+1 : 2m-1]$, $\mu(i) = 0$.

Firstly, μ is a distribution. Secondly, if we sample a random walk W starting from a state distributed according to μ—denoting the combined distribution by $\mathcal{D}_m$—, then, for all $j \in [2m-1]$,

$$\Pr_{W \sim \mathcal{D}_m}(W \text{ ends in } 0 \ \wedge \ \mathsf{w}_{\max}(W) = j) = \Pr_{W \sim \mathcal{D}_m}(W \text{ ends in } 0 \ \wedge \ \mathsf{w}_{\max}(W) = 2m - j)\,.$$

Finally,

$$\Pr_{W \sim \mathcal{D}_m}(W \text{ ends in } 0) = \frac{2m}{3m+1}.$$

We postpone the proof of this lemma to the end of this section to complete our definitions first. The proof can be skipped by the reader as it is not needed for understanding the rest of the paper.

Delayed and length-restricted random walks. Lemma 3.1 "symmetrizes" the distribution of maxima for a random walk W. We also need to ensure the distribution of the time of reaching the maxima is (essentially) the same across different choices for $\mathsf{w}_{\max}(W)$. Adding a "delay" to the walk fixes this issue as well.

A **delayed random walk** with delay $d \geqslant 1$ (also called d-delayed random walk) is defined similar to our original random walk, except that for its first $d-1$ time stamps, the walk stays in the same state and starts moving as before from time step d. Specifically, a d-delayed random walk $W = w_1, w_2, \cdots$, is such that

$$w_1 = \cdots = w_d \quad \text{and} \quad \text{for } t \geqslant d,\ w_{t+1} \text{ is chosen uniformly from } \{w_t + 1, w_t - 1\}\,,$$

and the walk as before terminates when hitting the state 0 or $2m$. We refer to the "non-delayed" part of the walk W as the *inner* random walk W_{in}, namely, $W_{in} = w_d, w_{d+1}, \cdots$.

For a d-delayed random walk W, we define

$$\mathsf{t}_{\max}(W) = (d-1) + \mathsf{t}_{\max}(W_{in}), \tag{4}$$

namely, the time the *inner* walk reached the eventual maximum (this distinction between inner and the entire walk is only relevant when the starting state is the eventual maximum; in this case, we consider $\mathsf{t}_{\max}(W) = d$, namely, the delay of the walk instead of time 1); $\mathsf{w}_{\max}(W)$ is defined as before which is equal to $\mathsf{w}_{\max}(W_{in})$ regardless.

Finally, we also need the length of the walks to be finite, which can be guaranteed (essentially) by Claim 2.3. We say that a delayed random walk is **ℓ-length restricted** iff it terminates within ℓ steps *after* spending its delayed time, i.e., reaches states 0 or $2m$ within $d + \ell$ steps assuming its delay was d (or alternatively, the length of its inner walk W_{in} is at most ℓ). By Claim 2.3, for large enough ℓ, a (delayed) random walk will be ℓ-length restricted with high probability (in m).

3.1 Proof of Lemma 3.1

We remark that the "right" way to prove a statement of the type Lemma 3.1 is to solve the program that maximizes $\Pr(W \text{ ends in } 0)$ subject to the symmetric constraints, under variables $\{\mu(i)\}_{i=1}^{2m-1}$ which are non-negative and sum to one. This becomes a linear programming problem which can then be solved explicitly (by considering its dual), to determine the values of $\mu(i)$'s. Nevertheless, since we have already solved this program and obtained $\mu(i)$'s, in the following proof we only verify the correctness of these variables.

We prove each part of Lemma 3.1 in the following.

μ forms a distribution. Clearly, all $\mu(i) \geqslant 0$ so we only need to prove $\sum_i \mu(i) = 1$. For this,

$$\begin{aligned}
\sum_i \mu(i) &= \gamma_m \cdot \sum_{i=1}^{m} \frac{2 \cdot (2m+1)}{(2m-i) \cdot (2m-i+1) \cdot (2m-i+2)} \\
&= \gamma_m \cdot \sum_{r=m}^{2m-1} \frac{2 \cdot (2m+1)}{r \cdot (r+1) \cdot (r+2)} && \text{(by reindexing the sum with } r = 2m-i) \\
&= \gamma_m \cdot \sum_{r=m}^{2m-1} (2m+1) \cdot \left(\frac{1}{r \cdot (r+1)} - \frac{1}{(r+1) \cdot (r+2)} \right) && \text{(by a direct calculation)} \\
&= \gamma_m \cdot (2m+1) \cdot \left(\frac{1}{m \cdot (m+1)} - \frac{1}{(2m) \cdot (2m+1)} \right) && \text{(by the telescoping sum)} \\
&= \gamma_m \cdot \frac{3m+1}{2m \cdot (m+1)} = 1,
\end{aligned}$$

by the definition of γ_m. Thus, μ is a distribution.

Symmetric property. To simplify the notation, in the following, we use '$w_T = 0$' to denote the event that the walk ends in 0 (instead of $2m$) and use '$w_1 = i$' for any integer i to mean the walk starts from the state i. For any $j \in [2m-1]$,

$$\begin{aligned}
\Pr_{W \sim \mathcal{D}_m} (w_T = 0 \ \wedge \ \mathsf{w}_{\max}(W) = j) &= \sum_{i=1}^{m} \mu(i) \cdot \Pr (w_T = 0 \ \wedge \ \mathsf{w}_{\max}(W) = j \mid w_1 = i) \\
&\qquad \text{(as the walk needs to start somewhere in } [m]) \\
&= \sum_{i=1}^{\min(m,j)} \mu(i) \cdot \Pr (w_T = 0 \ \wedge \ \mathsf{w}_{\max}(W) = j \mid w_1 = i) \\
&\qquad \text{(as otherwise starting from } i > j \text{ ensures } \mathsf{w}_{\max}(W) = j \text{ never happens}) \\
&= \sum_{i=1}^{\min(m,j)} \mu(i) \cdot \frac{i}{j \cdot (j+1)}, \qquad (5)
\end{aligned}$$

by Claim 2.2 since $1 \leqslant i \leqslant j \leqslant 2m$. We have the following claim on partial sums above.

Claim 3.2. *For any $j \in [m]$,*

$$P_j := \sum_{i=1}^{j} \mu(i) \cdot i = \gamma_m \cdot \frac{j \cdot (j+1)}{(2m-j) \cdot (2m-j+1)}.$$

Proof. The proof is by induction. For $j = 1$,

$$P_1 = \mu(1) = \gamma_m \cdot \frac{2 \cdot (2m+1)}{(2m-1) \cdot (2m) \cdot (2m+1)} = \gamma_m \cdot \frac{1 \cdot (1+1)}{(2m-1) \cdot (2m)},$$

matching the RHS of the claim. For $1 < j \leqslant m$, for the induction step, by the value of $\mu(j)$,

$$\begin{aligned} P_j = P_{j-1} + j \cdot \mu(j) &= \gamma_m \cdot \frac{(j-1) \cdot j}{(2m-j+1) \cdot (2m-j+2)} + j \cdot \gamma_m \cdot \frac{2 \cdot (2m+1)}{(2m-j) \cdot (2m-j+1) \cdot (2m-j+2)} \\ &= \gamma_m \cdot \frac{j \cdot (j-1) \cdot (2m-j) + 2j \cdot (2m+1)}{(2m-j) \cdot (2m-j+1) \cdot (2m-j+2)} \\ &\qquad \text{(by multiplying the first fraction with } (2m-j)/(2m-j)) \\ &= \gamma_m \cdot \frac{j \cdot (j+1)}{(2m-j) \cdot (2m-j+1)}, \end{aligned}$$

where the last equation can be verified using a direct calculation by verifying that

$$(2m-j+2) \cdot (j+1) = (j-1) \cdot (2m-j) + 2 \cdot (2m+1).$$

This concludes the proof. ∎ Claim 3.2

Plugging in Claim 3.2 in Eq (5), we have that for any $j \in [m]$,

$$\Pr_{W \sim \mathcal{D}_m} (w_T = 0 \ \wedge \ \mathsf{w}_{\max}(W) = j) = \frac{P_j}{j \cdot (j+1)} = \gamma_m \cdot \frac{1}{(2m-j) \cdot (2m-j+1)}.$$

On the other hand, for $j \in [m]$, we have $2m - j \geqslant m$ and by the max-term in Eq (5),

$$\Pr_{W \sim \mathcal{D}_m} (w_T = 0 \ \wedge \ \mathsf{w}_{\max}(W) = 2m-j) = \frac{P_m}{(2m-j) \cdot (2m-j+1)} = \gamma_m \cdot \frac{1}{(2m-j) \cdot (2m-j+1)},$$

since by Claim 3.2, $P_m = \gamma_m$. This proves the equality in the symmetric property.

Probability of W ending in 0. We calculate this directly as

$$\begin{aligned} \Pr_{W \sim \mathcal{D}_m} (w_T = 0) &= \sum_{i=1}^{m} \mu(i) \cdot \Pr_W (w_T = 0 \mid w_1 = i) && \text{(by the distribution of the start state)} \\ &= \sum_{i=1}^{m} \mu(i) \cdot \Pr_W (W \text{ reaches 0 before } 2m \mid w_1 = i) \\ &\qquad \text{(as otherwise the walk terminates in } 2m \text{ instead)} \\ &= \sum_{i=1}^{m} \mu(i) \cdot \frac{2m-i}{2m} && \text{(by Fact 2.1)} \\ &= 1 - \sum_{i=1}^{m} \frac{i \cdot \mu(i)}{2m} && \text{(since } \mu \text{ is a distribution so } \textstyle\sum_i \mu(i) = 1) \\ &= 1 - \frac{\gamma_m}{2m} \quad \text{(as the numerator in the sum-term is } P_m \text{ which is } \gamma_m \text{ by Claim 3.2)} \\ &= \frac{2m}{3m+1}, \end{aligned}$$

by the value of γ_m. This concludes the proof of Lemma 3.1. ∎

4 The Blueprint Construction

We provide our blueprint construction in this section, proving the following theorem.

Theorem (Restatement of Theorem 3)**.** *For any sufficiently large integer m, there exists a proper blueprint $\mathbb{G}_m$ with parameters $P = \text{poly}(m)$ and $C = m^{O(m)}$ such that*

$$value(\mathbb{G}_m) = \frac{2}{3} - \Theta\left(\frac{1}{m}\right).$$

We start by presenting the setup needed for this construction, followed by the construction itself and its properness proof, and finally calculate its value to conclude the proof of Theorem 3.

4.1 Setup

Let m be a large integer that is used to define all our other parameters for the blueprint. We define a blueprint $\mathbb{G} := \mathbb{G}(m)$ in this section. Define:

$L := m^4$, (used to determine the *length* of random walks)
$D := 2m \cdot L$, (used to determine the (range of) *delays* in random walks)
$P := L + D + 1$ (parameter P of the blueprint)
$C :=$ sufficiently large as function of m to be fixed later. (parameter C of the blueprint)

Recall that vertices of $\mathbb{G}$ are $\mathbb{L}, \mathbb{R}$ and they are labeled by $[C]^P$. We partition indices of P as follows:

$$P := (\texttt{start}, \texttt{delay}, \texttt{walk}) \quad \text{where}$$
$$\texttt{start} = 1, \quad \texttt{delay} = 2, \quad \text{and}, \quad \texttt{walk} = [3 : P].$$

We use these indices to associate a delayed random walk to vertices of the blueprint as follows.

Definition 4.1. Sample $x \in [C]^P$ uniformly at random and define a **delayed random walk** $W = W(x) = w_1, w_2, \cdots, w_T$ for some $T \leqslant D + L$ as follows[a].

1. $x_{\texttt{start}}$ determines the start state w_1 of the random walk W: we partition $[C]$ into intervals of proportional length $\mu(1), \mu(2), \ldots, \mu(m)$ (as defined in Lemma 3.1), and pick w_1 depending on which interval $x_{\texttt{start}}$ lands in.
2. $x_{\texttt{delay}}$ determines the delay of the delayed random walk W: we partition $[C]$ into D equal-length intervals and the delay of the random walk $d(x) = d$ if $x_{\texttt{delay}}$ lands in d-th interval.
3. $x_{\texttt{walk}}$ determines the inner walk: for any time $t \geqslant d(x)$, x_{t+2} determines the transition from w_t to w_{t+1} in W, where $w_{t+1} = w_t - 1$ if $x_{t+2} \in [1 : C/2]$ and $w_{t+1} = w_t + 1$ otherwise. The walk terminates whenever it hits 0 or $2m$ and the subsequent states remain the same.

We also define the **reflection walk** of x, denoted by $\overline{W}(x)$, exactly as $W(x)$ except for Line (3): we now increase the state if $x_i \in [1 : C/2]$ and decrease it otherwise (the exact opposite rule as in the definition of W).

[a] It is possible that the walk does not terminate and in that case we only consider its $D + L$ first states.

See Figure 2 for an illustration of this definition and further help with parsing it.

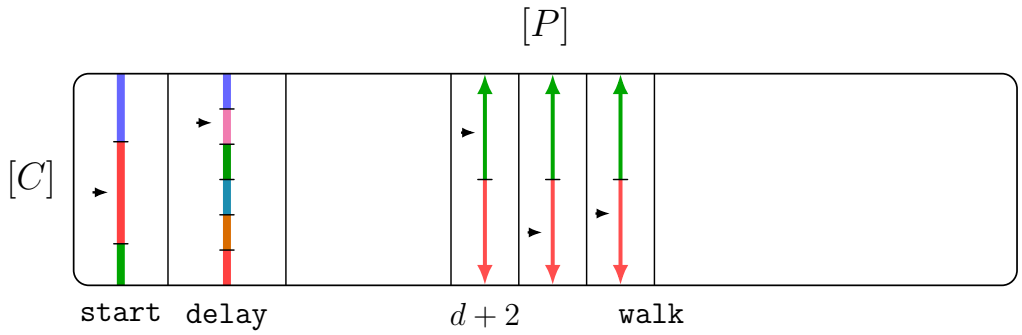


Figure 2: An illustration of the random walk $W(x)$ in Definition 4.1 when $m = 3$ and $D = 6$. The figure shows how the indices of $[P]$ are partitioned into $(\texttt{start}, \texttt{delay}, \texttt{walk})$. The intervals inside each box shows how $[C]$ is partitioned into different intervals. The tiny arrows in each box shows the value of a single point $x \in [C]^P$ on each relevant index. The walk $W(x)$ thus first considers $x_{\texttt{start}}$ and so chooses starting state 2. The delay d is set to 5 given $x_{\texttt{delay}}$. Finally, the walk waits until time d and then decide its next move based on the index $d + 2$ and so on; in this particular example, the walk goes to 3 next, then 2, and then 1.

Before getting to the main part, a quick remark is in order.

Remark 4.2. *We pick C such that*

- *$\mu(i) \cdot C$ is an integer for all $i \in [2m-1]$ (so the distribution of the start point in Line (1) matches μ);*
- *$D|C$ (so the distribution of the delay in Line (2) is uniform over $[D]$);*
- *C is even (so the distribution of the inner walk in Line (3) is a standard random walk).*

We can thus set

$$C = 2D \cdot \prod_{i=1}^{m} \text{denominator of } \mu(i)$$

and given the values of parameters $\mu(i)$ in Lemma 3.1 and since $D = \text{poly}(m)$, we have $C = m^{O(m)}$.

We use walks $W(x)$ for defining subsets of $\mathbb{L}$ and reflection walks $\overline{W}(x)$ for subsets in $\mathbb{R}$. Specifically, for any integer $j \in [2m-1]$ and time $t \in [D+L]$, we define

$$\mathbb{L}_{j,t} := \{\mathbb{L}(x) : W(x) \text{ is } L\text{-restricted, } W(x) \text{ reaches 0, } \mathtt{w}_{\max}(W(x)) = j, \text{ and } \mathtt{t}_{\max}(W(x)) = t\},$$
$$\mathbb{R}_{j,t} := \left\{\mathbb{R}(x) : \overline{W}(x) \text{ is } L\text{-restricted, } \overline{W}(x) \text{ reaches 0, } \mathtt{w}_{\max}(\overline{W}(x)) = j, \text{ and } \mathtt{t}_{\max}(\overline{W}(x)) = t\right\}.$$

These sets will play the key role in the definition of edges of the blueprint (they are essentially the endpoints of the edges).

We have the following immediate relation between the sizes of these sets.

Claim 4.3. *For all $j \in [2m-1]$ and $t \in [D+L]$, $|\mathbb{L}_{j,t}| = |\mathbb{R}_{j,t}|$.*

Proof. The only difference between $\mathbb{L}_{j,t}$ and $\mathbb{R}_{j,t}$ is in Line (3) of Definition 4.1: we thus have $\mathbb{L}(x) \in \mathbb{L}_{j,t}$ iff $\mathbb{R}(\overline{x}) \in \mathbb{R}_{j,t}$, where $\overline{x}$ is defined to be the same as x in $[P] \setminus \texttt{walk}$ and for $i \in \texttt{walk}$, $\overline{x}_i := C - x_i + 1$. Using the bijection between x's and $\overline{x}$'s, we can conclude the proof. ∎

The more interesting property is the near-equality of the sizes of $\mathbb{L}_{j,t}$ and $\mathbb{R}_{2m-j,t}$ (whenever $t \in [L : D]$); this is where we use most of the properties we incorporated in our random walks.

Lemma 4.4. *For all $j \in [2m-1]$ and $t \in [L : D]$,*

$$|\mathbb{R}_{2m-j,t}| - 2^{-m} \cdot \frac{C^P}{D} \leqslant |\mathbb{L}_{j,t}| \leqslant |\mathbb{R}_{2m-j,t}| + 2^{-m} \cdot \frac{C^P}{D}.$$

Proof. Define $W_{in}(x)$ as the inner walk of $W(x)$, which is the walk that starts at time $d(x)$ (the delay) and continues the same as W; we define $\overline{W}_{in}(x)$ similarly. In the following, when clear from the context, we drop (x) and write W to denote $W(x)$ (similarly for $W_{in}, \overline{W}$, etc.).

We first have

$$\begin{aligned}
\frac{|\mathbb{L}_{j,t}|}{C^P} &= \Pr_x\Big(W \text{ } L\text{-restricted, } W \text{ reaches 0, } \mathsf{w}_{\max}(W) = j,\, \mathsf{t}_{\max}(W) = t\Big) && \text{(by Definition 4.1)} \\
&= \Pr_x\Big(W_{in} \text{ } L\text{-restricted, } W_{in} \text{ reaches 0, } \mathsf{w}_{\max}(W_{in}) = j,\, \mathsf{t}_{\max}(W) = t\Big) \\
&\quad (L\text{-restrictedness, reaching 0, and the max-states are all the same between } W_{in} \text{ and } W) \\
&= \Pr_x\Big(W_{in} \text{ } L\text{-restricted, } W_{in} \text{ reaches 0, } \mathsf{w}_{\max}(W_{in}) = j,\, \mathsf{t}_{\max}(W_{in}) = t - (d(x)-1)\Big) \\
&\qquad\qquad \text{(by the definition of } \mathsf{t}_{\max}(W) \text{ in Eq (4))} \\
&= \frac{1}{D} \cdot \Pr_x\Big(W_{in} \text{ } L\text{-restricted, } W_{in} \text{ reaches 0, } \mathsf{w}_{\max}(W_{in}) = j\Big)
\end{aligned}$$

as $d(x)$ is independent of W_{in} and is uniform over $[D]$, and so the event $d(x) = t - \mathsf{t}_{\max}(W_{in}) + 1$ (conditioned on W_{in}) happens with probability $1/D$ as the RHS is in $[D]$ as well; this is because $\mathsf{t}_{\max}(W_{in}) \leqslant L$ (by L-restrictedness) and $t \in [L : D]$.

We can do the same exact analysis for $\mathbb{R}_{2m-j,t}$ and $\overline{W}$ as well to have

$$\frac{|\mathbb{R}_{2m-j,t}|}{C^P} = \frac{1}{D} \cdot \Pr_x\Big(\overline{W}_{in} \text{ } L\text{-restricted, } \overline{W}_{in} \text{ reaches 0, } \mathsf{w}_{\max}(\overline{W}_{in}) = 2m - j\Big).$$

Consider the expression in the RHS of either of these two terms. The distribution of W_{in} or $\overline{W}_{in}$ is exactly the same as the distribution of a random walk $\widetilde{W} \sim \mathcal{D}_m$ defined in Lemma 3.1. Hence,

$$\begin{aligned}
\frac{|\mathbb{L}_{j,t}|}{C^P} &= \frac{1}{D} \cdot \Pr_{\widetilde{W}}\Big(\widetilde{W} \text{ } L\text{-restricted, } \widetilde{W} \text{ reaches 0, } \mathsf{w}_{\max}(\widetilde{W}) = j\Big) \\
&\geqslant \frac{1}{D} \cdot \left(\Pr_{\widetilde{W}}\Big(\widetilde{W} \text{ reaches 0, } \mathsf{w}_{\max}(\widetilde{W}) = j\Big) - \Pr_{\widetilde{W}}\Big(\widetilde{W} \text{ not } L\text{-restricted}\Big)\right) \\
&\qquad\qquad \text{(as } \Pr(A \wedge B) \geqslant \Pr(B) - \Pr(\overline{A})) \\
&\geqslant \frac{1}{D} \cdot \left(\Pr_{\widetilde{W}}\Big(\widetilde{W} \text{ reaches 0, } \mathsf{w}_{\max}(\widetilde{W}) = j\Big) - 2^{-m}\right) \qquad \text{(as } L = m^4 \text{ and by Claim 2.3)} \\
&= \frac{1}{D} \cdot \left(\Pr_{\widetilde{W}}\Big(\widetilde{W} \text{ reaches 0, } \mathsf{w}_{\max}(\widetilde{W}) = 2m - j\Big) - 2^{-m}\right) \\
&\qquad\qquad \text{(by the symmetry property of Lemma 3.1)} \\
&\geqslant \frac{1}{D} \cdot \left(\Pr_{\widetilde{W}}\Big(\widetilde{W} \text{ } L\text{-restricted, } \widetilde{W} \text{ reaches 0, } \mathsf{w}_{\max}(\widetilde{W}) = 2m - j\Big) - 2^{-m}\right) \\
&\qquad\qquad \text{(as } \Pr(A) \geqslant \Pr(A \wedge B)) \\
&= \frac{|\mathbb{R}_{2m-j,t}|}{C^P} - \frac{2^{-m}}{D}.
\end{aligned}$$

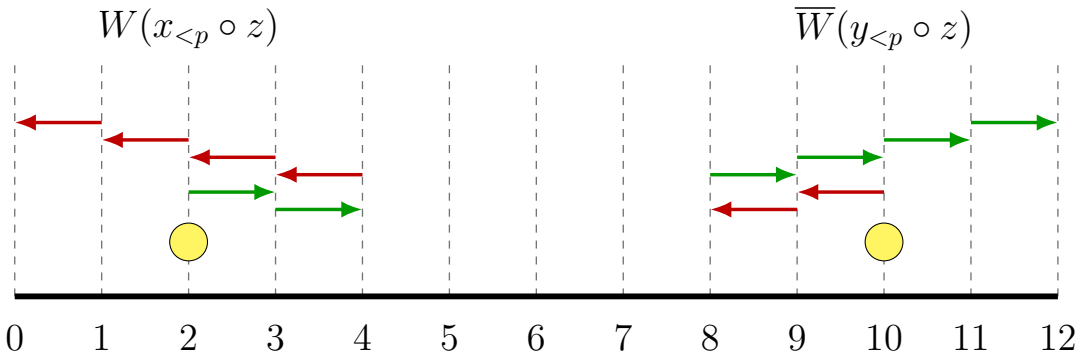


Figure 3: An illustration of the proof of the ban constraint (Claim 4.6) for $\mathbb{G}$ with $m = 6$, $j = 2$, and $z \approx (\uparrow, \uparrow, \downarrow, \downarrow, \downarrow, \downarrow)$, which by that we mean the $\uparrow$ indices are in $[(C/2)+1 : C]$ and $\downarrow$ ones are in $[1 : C/2]$. Suppose the choices of $x_{<p}$ and $y_{<p}$ already "bring" the walks $W(x_{<p} \circ z)$ and $\overline{W}(y_{<p} \circ z)$ to states 2 and 10, respectively. From there, $W(x_{<p} \circ z)$, increases the state by two and then decreases it by four, whereas by reflection, $\overline{W}(y_{<p} \circ z)$ does the exact opposite; thus, $W(x_{<p} \circ z)$ reaches state 0 iff $\overline{W}(y_{<p} \circ z)$ reaches $12 (= 2m)$. As such, at most one pair of these vertices can be matched in $\mathbb{G}$ (only vertices whose walk reaches 0 are matched regardless of being in $\mathbb{L}$ or $\mathbb{R}$).

Applying the same argument starting from $\mathbb{R}_{2m-j,t}$ then implies that

$$|\mathbb{R}_{2m-j,t}| - 2^{-m} \cdot \frac{C^P}{D} \leqslant |\mathbb{L}_{j,t}| \leqslant |\mathbb{R}_{2m-j,t}| + 2^{-m} \cdot \frac{C^P}{D} \qquad \blacksquare$$

4.2 The Construction

We are now ready to define the edges of the blueprint $\mathbb{G} = (\mathbb{L}, \mathbb{R}, \mathbb{E})$ and prove its properness. As stated earlier, both vertex-sets $\mathbb{L}$ and $\mathbb{R}$ are labeled by $[C]^P$. The edges in $\mathbb{E}$ are defined as follows:

- For any $j \in [2m-1]$ and $t \in [L : D]$, we add a matching $\mathbb{M}_{j,t}$ of size $s_{j,t} := \min(|\mathbb{L}_{j,t}|, |\mathbb{R}_{2m-j,t}|)$ between two arbitrarily-chosen $s_{j,t}$-subsets of $\mathbb{L}_{j,t}$ and $\mathbb{R}_{2m-j,t}$. Moreover, the matching $\mathbb{M}_{j,t}$ is added as part of $\mathbb{E}^{(p)}$ for $p = t + 2$.

We need to check that the blueprint $\mathbb{G}$ is proper, which is handled by the following two claims.

Claim 4.5 ("Matching constraint")**.** *The edge-set $\mathbb{E}$ is a matching.*

Proof. By definition, the sets $\mathbb{L}_{j,t}$ are disjoint for different values of j (the maximum of a random walk is a unique state) as well as t (the *first* time reaching the eventual maximum is unique). Same is true for $\mathbb{R}_{2m-j,t}$ as well, thus $\mathbb{E} = \cup_{p \in [P]} \mathbb{E}^{(p)}$ itself is a matching also. $\blacksquare$

Claim 4.6 ("Ban constraint")**.** *The edge-set $\mathbb{E}$ respects the ban constraint.*

Proof. Consider an edge $\mathbb{e} \in \mathbb{E}^{(p)}$ for $p = t + 2$ for some arbitrary $t \in [L : D]$ and suppose $\mathbb{e}$ is between some $\mathbb{L}(x) \in \mathbb{L}_{j,t}$ and $\mathbb{R}(y) \in \mathbb{R}_{2m-j,t}$ for $x, y \in [C]^P$. Recall all edges of $\mathbb{G}$ are of this form.

By Definition 1.2, the ban constraint requires that for all $z \in [C]^{P-p+1}$, at most one of $\mathbb{L}(x_{<p} \circ z)$ and $\mathbb{R}(y_{<p} \circ z)$ is matched in $\mathbb{E}$. We show that this is the case for every fixed $z \in [C]^{P-p+1}$. Figure 3 can be helpful with providing an illustration of this step of the proof.

Consider the walk $W(x_{<p} \circ z)$. Fixing $x_{<p}$ fixes the starting state of the walk (by fixing $x_{\texttt{start}}$), the delay of the walk (by fixing $x_{\texttt{delay}}$) and the states until time t of the walk. By the definition of $\mathbb{L}(x) \in \mathbb{L}_{j,t}$, we know that at the time t, the walk $W(x_{<p} \circ z)$ will also be at the state j. Thus, for $\mathbb{L}(x_{<p} \circ z)$ to be matched, we need the walk $W(x_{<p} \circ z)$ to reach the state 0 or equivalently, we need the walk $W_z := w_t(= j), w_{t+1}, \cdots$, to reach 0 starting from j. Note that the entire movement of the walk W_z is determined by z.

On the other hand, consider the reflection walk $\overline{W}(y_{<p} \circ z)$. Again, fixing $y_{<p}$ and the fact $\mathbb{R}(y) \in \mathbb{R}_{2m-j,t}$, in particular means that this walk is at state $2m - j$ at time t. Consider the continuation of this walk as $\overline{W}_z := \bar{w}_t(= 2m - j), \bar{w}_{t+1}, \cdots$, starting from $2m - j$ which is fully determined by z. Given $\overline{W}_z$ is a reflection of W_z, if W_z starting from j reaches 0, then, $\overline{W}_z$ starting from $2m - j$ will reach $2m$. But, that means that the walk $\overline{W}(y_{<p} \circ z)$ ends in the state $2m$ which forces $\mathbb{R}(y_{<p} \circ z)$ to never be matched in the blueprint.

Combining the above, we have that between the two vertices $\mathbb{L}(x_{<p} \circ z)$ and $\mathbb{R}(y_{<p} \circ z)$, at most one is matched in the blueprint, respecting the ban constraint. ∎

We thus established that the blueprint $\mathbb{G}$ is indeed a proper blueprint.

4.3 Value Calculation

The last step is to calculate the value of the blueprint which is $|\mathbb{E}| / C^P$.

Lemma 4.7. *The value of blueprint $\mathbb{G} = \mathbb{G}_m$ is*

$$value(\mathbb{G}) \geqslant \left(1 - \frac{1}{m}\right) \cdot \frac{2m}{3m+1} - 3m \cdot 2^{-m},$$

Proof. Given $\mathbb{E}$ is a matching, and by construction,

$$\begin{aligned}
value(\mathbb{G}) &= \frac{|\mathbb{E}|}{C^P} && \text{(by Definition 1.3)} \\
&= \sum_{j=1}^{2m-1} \sum_{t=L}^{D} \frac{\min\left(|\mathbb{L}_{j,t}|, |\mathbb{R}_{2m-j,t}|\right)}{C^P} && \text{(by the definition of edges)} \\
&\geqslant \sum_{j=1}^{2m-1} \sum_{t=L}^{D} \left(\frac{|\mathbb{L}_{j,t}|}{C^P} - 2^{-m} \cdot \frac{1}{D}\right) && \text{(by Lemma 4.4)} \\
&\geqslant \left(\sum_{j=1}^{2m-1} \sum_{t=L}^{D} \frac{|\mathbb{L}_{j,t}|}{C^P}\right) - 2m \cdot 2^{-m}.
\end{aligned}$$

We can now bound the inner sum separately as follows:

$$\begin{aligned}
\sum_{j=1}^{2m-1} \sum_{t=L}^{D} \frac{|\mathbb{L}_{j,t}|}{C^P} &= \sum_{j=1}^{2m-1} \sum_{t=L}^{D} \Pr\Big(W \text{ } L\text{-restricted, } W \text{ reaches 0, } \mathsf{w}_{\max}(W) = j, \mathsf{t}_{\max}(W) = t\Big) \\
&\qquad \text{(by the definition of } \mathbb{L}_{j,t}) \\
&= \sum_{j=1}^{2m-1} \Pr\Big(W \text{ } L\text{-restricted, } W \text{ reaches 0, } \mathsf{w}_{\max}(W) = j, \mathsf{t}_{\max}(W) \in [L:D]\Big) \\
&\qquad \text{(as the events in the sum are all disjoint)}
\end{aligned}$$

$$\geqslant \sum_{j=1}^{2m-1} \Pr\Big(W \text{ } L\text{-restricted, } W \text{ reaches 0, } \mathsf{w}_{\max}(W) = j,\ d(x) \in [L : D-L]\Big)$$

(since W is L-restricted, having delay in $[L : D-L]$ ensures $\mathsf{t}_{\max}(W) \in [L : D]$)

$$\geqslant \frac{D-2L}{D} \cdot \sum_{j=1}^{2m-1} \Pr\Big(W \text{ } L\text{-restricted, } W \text{ reaches 0, } \mathsf{w}_{\max}(W) = j\Big)$$

(as $d(x)$ is uniform independently on $[D]$ and $|[L : D-L]| = D-2L+1 > D-2L$)

$$= \frac{D-2L}{D} \cdot \Pr\Big(W \text{ } L\text{-restricted, } W \text{ reaches 0}\Big)$$

(the sum is over all disjoint possibilities of reaching 0)

$$\geqslant \frac{D-2L}{D} \cdot \Big(\Pr\Big(\widetilde{W} \text{ reaches 0}\Big) - 2^{-m}\Big)$$

(by Claim 2.3 and using $\widetilde{W}$ as a generic random walk)

$$= \frac{D-2L}{D} \cdot \left(\frac{2m}{3m+1} - 2^{-m}\right) \qquad \text{(by Lemma 3.1)}$$

$$\geqslant \left(1 - \frac{1}{m}\right) \cdot \frac{2m}{3m+1} - 2^{-m}. \qquad \text{(as } D = 2m \cdot L\text{)}$$

By plugging in this into the earlier bound, we have,

$$\mathit{value}(\mathbb{G}) \geqslant \left(1 - \frac{1}{m}\right) \cdot \frac{2m}{3m+1} - 3m \cdot 2^{-m},$$

concluding the proof. ∎

The lower bound in Theorem 3 now follows immediately from Lemma 4.7 and the properness of the blueprint established in Claims 4.5 and 4.6. The upper bound is simply because

$$\mathit{value}(\mathbb{G}_m) \leqslant \Pr_{W \sim \mathcal{D}_m}(W \text{ reaches 0}) = \frac{2m}{3m+1},$$

by Lemma 3.1, as every matched vertex in $\mathbb{L}$ belongs to some $\mathbb{L}_{j,t}$ and hence its walk has reached 0.

This concludes the proof of Theorem 3.

Acknowledgement

We are thankful to Aaron Bernstein, Gunadi Gani, Lap Chi Lau, Euiwoong Lee, and Thatchaphol Saranurak for helpful discussions on blueprints, and to Lap Chi Lau and Janani Sundaresan for many helpful comments on the presentation of an earlier version of the paper. We are also grateful to Trever Brown for providing us with computational resources for generating blueprints efficiently in the early stages of this project and to David Woodruff for running multiple LLM prompts on our behalf on Google's Gemini-based models.

Sepehr Assadi is additionally grateful to Soheil Behnezhad, Aaron Bernstein, Niv Buchbinder, Deeparnab Chakrabarty, Michael Kapralov, Sanjeev Khanna, Christian Konrad, Lap Chi Lau, Thatchaphol Saranurak, and David Wajc for various discussions on semi-streaming matching over the years.

AI Acknowledgement

We have used Google's Gemini-based models, GPT 5.5 Pro, GPT 5.6 Sol, Claude Opus, and Claude Fable as part of our research. Gemini-based models were used briefly during the preliminary stages for a small number of prompts using only the definitions of blueprints, without additional project context, and their outputs have not been used in the rest of the project. The remaining systems were routinely supplied with the authors' evolving research notes and ideas, and have been used for brainstorming and technical assistance. Specifically, the authors provided an optimal solution to a *relaxation* of blueprints (a generalization of the "density"-argument in Section A), and a combination of Claude Fable and GPT 5.6 Sol gave the idea for using random walks to take this solution from the relaxations to blueprints. These random walks formed the crux of our blueprints. GPT 5.6 Sol also assisted with the proof of Lemma 3.1 by solving the optimization problem after authors' directive to formulate it as a linear program. The proof in the paper is different than the AI-generated proof and is our own as we could simply use the provided variable values and directly prove they satisfy the constraints and the bound on the objective.

AI tools were also used to assist in proofreading and creating figures from given drawings. No AI-generated text appears in the paper: the authors formulated and wrote all the final mathematical content themselves, including all the statements and proofs. The authors take full responsibility for all content.

References

[AB21] S. Assadi and S. Behnezhad. Beating two-thirds for random-order streaming matching. In *48th International Colloquium on Automata, Languages, and Programming, ICALP 2021, July 12-16, 2021, Glasgow, Scotland (Virtual Conference)*, volume 198 of *LIPIcs*, pages 19:1–19:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. 1

[ABB+19] S. Assadi, M. Bateni, A. Bernstein, V. S. Mirrokni, and C. Stein. Coresets meet EDCS: algorithms for matching and vertex cover on massive graphs. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019*, pages 1616–1635. SIAM, 2019. 5

[ABKL23] S. Assadi, S. Behnezhad, S. Khanna, and H. Li. On regularity lemma and barriers in streaming and dynamic matching. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, pages 131–144. ACM, 2023. 1

[ADKN23] C. Alexandru, P. Dvorák, C. Konrad, and K. K. Naidu. Improved weighted matching in the sliding window model. In *40th International Symposium on Theoretical Aspects of Computer Science, STACS 2023, Hamburg, Germany, March 7-9, 2023*, volume 254 of *LIPIcs*, pages 6:1–6:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. 1

[AJX26] S. Assadi, M. Jiang, and M. Xiang. Semi-streaming matching in a single pass: A new framework for lower bounds via blueprints. In *Proceedings of the 58th Annual ACM Symposium on Theory of Computing, STOC 2026, Salt Lake City, UT, USA, June 22-26, 2026*, pages 979–990. ACM, 2026. ii, 1, 2, 3, 5, 20

[AK17] S. Assadi and S. Khanna. Randomized composable coresets for matching and vertex cover. In *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA 2017, Washington DC, USA, July 24-26, 2017*, pages 3–12. ACM, 2017. 5

[Beh21] S. Behnezhad. *Modern Large-Scale Algorithms for Classical Graph Problems*. PhD thesis, University of Maryland, College Park, MD, USA, 2021. 1

[Ber24] A. Bernstein. Improved bounds for matching in random-order streams. *Theory Comput. Syst.*, 68(4):758–772, 2024. 1

[CJMM17] G. Cormode, H. Jowhari, M. Monemizadeh, and S. Muthukrishnan. The sparse awakens: Streaming algorithms for matching size estimation in sparse graphs. In *25th Annual European Symposium on Algorithms, ESA 2017, Vienna, Austria, September 4-6, 2017*, volume 87 of *LIPIcs*, pages 29:1–29:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. 1

[CK14] A. Chakrabarti and S. Kale. Submodular maximization meets streaming: Matchings, matroids, and more. In *Integer Programming and Combinatorial Optimization - 17th International Conference, IPCO 2014, Bonn, Germany, June 23-25, 2014. Proceedings*, volume 8494 of *Lecture Notes in Computer Science*, pages 210–221. Springer, 2014. 1

[DGR25] M. Derakhshan, A. Ghasemi, and R. Rajaraman. One-way communication complexity of minimum vertex cover in general graphs. In *52nd International Colloquium on Automata, Languages, and Programming, ICALP 2025, July 8-11, 2025, Aarhus, Denmark*, volume 334 of *LIPIcs*, pages 66:1–66:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. 1, 5

[EHM16] H. Esfandiari, M. Hajiaghayi, and M. Monemizadeh. Finding large matchings in semi-streaming. In *IEEE International Conference on Data Mining Workshops, ICDM Workshops 2016, December 12-15, 2016, Barcelona, Spain*, pages 608–614. IEEE Computer Society, 2016. 1

[ELSW13] L. Epstein, A. Levin, D. Segev, and O. Weimann. Improved bounds for online preemptive matching. In *30th International Symposium on Theoretical Aspects of Computer Science, STACS 2013, February 27 - March 2, 2013, Kiel, Germany*, volume 20 of *LIPIcs*, pages 389–399. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2013. 5

[FKM+05] J. Feigenbaum, S. Kannan, A. McGregor, S. Suri, and J. Zhang. On graph problems in a semi-streaming model. *Theor. Comput. Sci.*, 348(2-3):207–216, 2005. 1

[FMU22] M. Fischer, S. Mitrovic, and J. Uitto. Deterministic $(1+\epsilon)$-approximate maximum matching with poly$(1/\epsilon)$ passes in the semi-streaming model and beyond. In *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*, pages 248–260. ACM, 2022. 1

[FNSZ22] M. Feldman, A. Norouzi-Fard, O. Svensson, and R. Zenklusen. Submodular maximization subject to matroid intersection on the fly. In *30th Annual European Symposium on Algorithms, ESA 2022, Berlin/Potsdam, Germany, September 5-9, 2022*, volume 244 of *LIPIcs*, pages 52:1–52:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. 1

[FS22] M. Feldman and A. Szarf. Maximum matching sans maximal matching: A new approach for finding maximum matchings in the data stream model. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2022, University of Illinois, Urbana-Champaign, USA (Virtual*

Conference), September 19-21, 2022, volume 245 of *LIPIcs*, pages 33:1–33:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. 1

[Gam21] B. Gamlath. *How to Match in Modern Computational Settings*. PhD thesis, EPFL, Switzerland, 2021. 1

[GKK12] A. Goel, M. Kapralov, and S. Khanna. On the communication and streaming complexity of maximum bipartite matching. In *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2012, Kyoto, Japan, January 17-19, 2012*, pages 468–485. SIAM, 2012. 1, 5

[GKM+19] B. Gamlath, M. Kapralov, A. Maggiori, O. Svensson, and D. Wajc. Online matching with general arrivals. In *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019*, pages 26–37. IEEE Computer Society, 2019. 5

[GKMS19] B. Gamlath, S. Kale, S. Mitrovic, and O. Svensson. Weighted matchings via unweighted augmentations. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 491–500. ACM, 2019. 1

[GKRS20] P. Garg, S. Kale, L. Rohwedder, and O. Svensson. Robust algorithms under adversarial injections. In *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, Saarbrücken, Germany (Virtual Conference), July 8-11, 2020*, volume 168 of *LIPIcs*, pages 56:1–56:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. 1

[HPT+19] Z. Huang, B. Peng, Z. G. Tang, R. Tao, X. Wu, and Y. Zhang. Tight competitive ratios of classic matching algorithms in the fully online model. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019*, pages 2875–2886. SIAM, 2019. 5

[Kap13] M. Kapralov. Better bounds for matchings in the streaming model. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2013, New Orleans, Louisiana, USA, January 6-8, 2013*, pages 1679–1697, 2013. 1, 5

[Kap21] M. Kapralov. Space lower bounds for approximating maximum matching in the edge arrival model. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021*, pages 1874–1893. SIAM, 2021. 1, 5

[KMM12] C. Konrad, F. Magniez, and C. Mathieu. Maximum matching in semi-streaming with few passes. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques - 15th International Workshop, APPROX 2012, and 16th International Workshop, RANDOM 2012, Cambridge, MA, USA, August 15-17, 2012. Proceedings*, volume 7408 of *Lecture Notes in Computer Science*, pages 231–242. Springer, 2012. 1

[KN21] C. Konrad and K. K. Naidu. On two-pass streaming algorithms for maximum bipartite matching. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2021, University of Washington, Seattle,*

Washington, USA (Virtual Conference), August 16-18, 2021, volume 207 of *LIPIcs*, pages 19:1–19:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. 1

[KR03] S. Khot and O. Regev. Vertex cover might be hard to approximate to within $2-\varepsilon$. In *18th Annual IEEE Conference on Computational Complexity (Complexity 2003), 7-10 July 2003, Aarhus, Denmark*, page 379. IEEE Computer Society, 2003. 5

[KS26] P. Kiss and M. Sharifi. Online preemptive matching revisited. In *53rd International Colloquium on Automata, Languages, and Programming, ICALP 2026, Royal Holloway, University of London, Egham, United Kingdom, July 7-10, 2026*, volume 374 of *LIPIcs*, pages 128:1–128:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2026. 5

[KT17] S. Kale and S. Tirodkar. Maximum matching in two, three, and a few more passes over graph streams. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2017, Berkeley, CA, USA, August 16-18, 2017*, volume 81 of *LIPIcs*, pages 15:1–15:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. 1

[KVV90] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. An optimal algorithm for on-line bipartite matching. In *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing, May 13-17, 1990, Baltimore, Maryland, USA*, pages 352–358. ACM, 1990. 5

[LP26] D. A. Levin and Y. Peres. *Markov chains and mixing times*. American Mathematical Society, 2026. 4, 6

[McG13] A. McGregor. Open Problems in Sublinear Algorithms, Problem 60: Single-Pass Unweighted Matchings. https://sublinear.info/60, 2013. Suggested by Andrew McGregor; accessed July 14, 2026. 1

[McG14] A. McGregor. Graph stream algorithms: a survey. *SIGMOD Rec.*, 43(1):9–20, 2014. 1

[Nai24] K. K. Naidu. *Streaming Maximum Matching in a Few Passes: Algorithms and Lower Bounds*. PhD thesis, University of Bristol, 2024. 1

[Tir18] S. Tirodkar. Deterministic algorithms for maximum matching on general graphs in the semi-streaming model. In *38th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2018, Ahmedabad, India, December 11-13, 2018*, volume 122 of *LIPIcs*, pages 39:1–39:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. 1

[Waj21] D. Wajc. *Matching Theory Under Uncertainty*. PhD thesis, Carnegie Mellon University, USA, 2021. 1

Appendix

A *Upper Bounding* the Value of Blueprints

For completeness and to provide further intuition about blueprints, we prove an optimal upper bound for Eq (1). As stated earlier, this bound also follows from Theorem 2 of [AJX26] and the greedy algorithm for the semi-streaming matching problem.

Lemma A.1. $\sup\{\mathit{value}(\mathbb{G}) : \mathbb{G} \text{ a proper blueprint}\} \leqslant \frac{2}{3}$.

Proof. Fix any blueprint $\mathbb{G} = (\mathbb{L}, \mathbb{R}, \mathbb{E})$ with parameters (P, C). Define $\mathbb{X}_L$ (resp. $\mathbb{X}_R$) to be the matched vertices in $\mathbb{L}$ (resp. $\mathbb{R}$). We use $\nu := \mathit{value}(\mathbb{G})$ for the simplicity of notation. We will prove that $\nu \leqslant 2/3$ (throughout we use the fact that $\nu > 1/2$ to simplify some definitions).

Consider $\mathbb{L}$ first. For any vertex $\mathbb{L}(x)$ for $x \in [C]^P$, define

$$D(\mathbb{L}(x)) := \min_{p \in [P+1]} \Pr_{z \in_R [C]^P} \left(\mathbb{L}(z) \in \mathbb{X}_L \mid z_{<p} = x_{<p}\right).$$

For each $p \in [P+1]$, the corresponding min-term in the RHS measures the *density* of $\mathbb{X}_L$ inside vertices consistent with $x_{<p}$. Specifically, for $p = 1$, that term is simply $\Pr(\mathbb{L}(z) \in \mathbb{X}_L) = \nu$ and for $p = P+1$ it is either 1 or 0 depending on whether $\mathbb{L}(x) \in \mathbb{X}_L$ or not.

We define this analogously for $\mathbb{R}$ by replacing $\mathbb{X}_L$ with $\mathbb{X}_R$ and considering $\mathbb{R}(x)$ for $x \in [C]^P$.

Claim A.2. *For any edge* $(\mathbb{L}(x), \mathbb{R}(y)) \in \mathbb{E}$, $D(\mathbb{L}(x)) + D(\mathbb{R}(y)) \leqslant 1$.

Proof. Fix an edge $(\mathbb{L}(x), \mathbb{R}(y)) \in \mathbb{E}^{(p)}$ for some $p \in [P]$. By the ban constraint (Definition 1.2), for each $z \in [C]^{P-p+1}$, at most one of vertices $\mathbb{L}(x_{<p} \circ z)$ and $\mathbb{R}(y_{<p} \circ z)$ can be matched in $\mathbb{E}$. Thus,

$$\Pr_{w \in_R [C]^P} \left(\mathbb{L}(w) \in \mathbb{X}_L \mid w_{<p} = x_{<p}\right) + \Pr_{w \in_R [C]^P} \left(\mathbb{R}(w) \in \mathbb{X}_R \mid w_{<p} = y_{<p}\right) \leqslant 1.$$

Taking the minimum on each term can only decrease the LHS, implying the claim. ∎ Claim A.2

By Claim A.2, we have

$$\Pr_{(\mathbb{L}(x), \mathbb{R}(y)) \in_R \mathbb{E}} \left(D(\mathbb{L}(x)) > \frac{1}{2} \ \wedge \ D(\mathbb{R}(y)) > \frac{1}{2}\right) = 0,$$

which, by union bound and since $\mathbb{E}$ is a matching, implies that

$$\begin{aligned} 1 &= \Pr_{(\mathbb{L}(x), \mathbb{R}(y)) \in_R \mathbb{E}} \left(D(\mathbb{L}(x)) \leqslant \frac{1}{2} \vee D(\mathbb{R}(y)) \leqslant \frac{1}{2}\right) \\ &\leqslant \Pr_{\mathbb{L}(x) \in_R \mathbb{X}_L} \left(D(\mathbb{L}(x)) \leqslant \frac{1}{2}\right) + \Pr_{\mathbb{R}(y) \in_R \mathbb{X}_R} \left(D(\mathbb{R}(y)) \leqslant \frac{1}{2}\right). \end{aligned} \tag{6}$$

Claim A.3. $\Pr_{\mathbb{L}(x) \in_R \mathbb{X}_L} \left(D(\mathbb{L}(x)) \leqslant \frac{1}{2}\right) \leqslant \frac{1-\nu}{\nu}$.

Proof. We say $x \in [C]^P$ is **bad** if $D(\mathbb{L}(x)) \leqslant 1/2$. We further say x is ***p*-bad** for $p \in [2 : P+1]$ iff p is the *smallest* index where

$$\Pr_{z \in_R [C]^P} \left(\mathbb{L}(z) \in \mathbb{X}_L \mid z_{<p} = x_{<p}\right) \leqslant \frac{1}{2}.$$

Any bad vertex is p-bad for a unique choice of $p \in [2 : P+1]$. We have

$$\Pr_{\mathbb{L}(x)\in_R \mathbb{X}_L}\left(D(\mathbb{L}(x)) \leqslant \frac{1}{2}\right) = \Pr_{x\in_R [C]^P}\left(x \text{ is bad} \mid \mathbb{L}(x) \in \mathbb{X}_L\right)$$
$$= \frac{\Pr_{x\in_R [C]^P}\left(x \text{ is bad} \wedge \mathbb{L}(x) \in \mathbb{X}_L\right)}{\Pr_{x\in_R [C]^P}\left(\mathbb{L}(x) \in \mathbb{X}_L\right)}. \tag{7}$$

The denominator in the RHS is ν by definition. We now bound the numerator.

$$\Pr_{x\in_R [C]^P}\left(x \text{ is bad} \wedge \mathbb{L}(x) \in \mathbb{X}_L\right) = \sum_{p=2}^{P+1} \Pr_{x\in_R [C]^P}\left(x \text{ is } p\text{-bad} \wedge \mathbb{L}(x) \in \mathbb{X}_L\right)$$
(as a bad x is p-bad for a unique p)
$$= \sum_{p=2}^{P+1} \Pr_{x\in_R [C]^P}\left(x \text{ is } p\text{-bad}\right) \cdot \Pr_{x\in_R [C]^P}\left(\mathbb{L}(x) \in \mathbb{X}_L \mid x \text{ is } p\text{-bad}\right)$$
$$\leqslant \sum_{p=2}^{P+1} \Pr_{x\in_R [C]^P}\left(x \text{ is } p\text{-bad}\right) \cdot \frac{1}{2} \qquad \text{(explained below)}$$
$$= \frac{1}{2} \Pr_{x\in_R [C]^P}\left(x \text{ is bad}\right).$$
(again, since p-bad events partition the bad event)

The inequality above holds because conditioning on x being p-bad requires fixing $x_{<p}$, and so by sampling the remaining coordinates and the definition of p-bad, we know $\mathbb{L}(x)$ belongs to $\mathbb{X}_L$ with probability at most $1/2$.

Finally, we can upper bound the probability that x is bad (rather crudely) as follows:

$$\nu = \Pr_{x\in_R [C]^P}\left(\mathbb{L}(x) \in \mathbb{X}_L\right) = \Pr_{x\in_R [C]^P}\left(\mathbb{L}(x) \in \mathbb{X}_L \wedge x \text{ is bad}\right) + \Pr_{x\in_R [C]^P}\left(\mathbb{L}(x) \in \mathbb{X}_L \wedge x \text{ is not bad}\right)$$
$$\leqslant \frac{1}{2} \cdot \Pr_{x\in_R [C]^P}\left(x \text{ is bad}\right) + \Pr_{x\in_R [C]^P}\left(x \text{ is not bad}\right) \qquad \text{(by the previous equation)}$$
$$\leqslant \frac{1}{2} \cdot \Pr_{x\in_R [C]^P}\left(x \text{ is bad}\right) + 1 - \Pr_{x\in_R [C]^P}\left(x \text{ is bad}\right).$$

Therefore, we have,

$$\Pr_{x\in_R [C]^P}\left(x \text{ is bad}\right) \leqslant 2 \cdot (1-\nu).$$

Plugging these bounds in Eq (7) implies that

$$\Pr_{\mathbb{L}(x)\in_R \mathbb{X}_L}\left(D(\mathbb{L}(x)) \leqslant \frac{1}{2}\right) \leqslant \frac{1-\nu}{\nu},$$

concluding the proof. ∎ Claim A.3

The same exact proof as Claim A.3 also holds for vertices in $\mathbb{R}$ and $\mathbb{X}_R$ by symmetry. Plugging in these bounds in Eq (6), we have,

$$1 \leqslant \Pr_{\mathbb{L}(x)\in_R \mathbb{X}_L}\left(D(\mathbb{L}(x)) \leqslant \frac{1}{2}\right) + \Pr_{\mathbb{R}(y)\in_R \mathbb{X}_R}\left(D(\mathbb{R}(y)) \leqslant \frac{1}{2}\right) \leqslant 2 \cdot \frac{1-\nu}{\nu},$$

which in turn implies that $\nu \leqslant 2/3$, concluding the proof of Lemma A.1. ∎